Java Streams — 50 Practice Questions (Blind-75 style)

Solve each: (1) as a loop first, (2) convert to stream, (3) name the pattern # from the playbook before you start. Answer key (pattern + collector only, not full code) is at the bottom — don't peek until you've attempted it.

Assume a `List<Employee>` (id, name, dept, salary, age) and a `List<String> words` are available unless stated otherwise.

Group / Count / Aggregate (Patterns 1–3)

1. Group employees by department.
2. Count employees per department.
3. Find average salary per department.
4. Find the highest-paid employee in each department.
5. Find total salary cost per department.
6. Group words by their length.
7. Group employees by decade of age (20s, 30s, 40s...).
8. Find the department with the most employees.
9. Group students by grade (A/B/C) computed from their score.
10. Count how many employees are above/below the company average salary.

Order-preserving group (Pattern 4)

11. Find the first repeating character in a string.
12. Find the first non-repeating character in a string.
13. Given a list of numbers, return them grouped by parity (even/odd) but preserve first-seen order within each group.

Lookup / toMap (Pattern 5)

14. Build a map of employee id → employee name.
15. Build a map of department → total salary (handle duplicate depts with a merge function).
16. From two lists of equal length (names, ages), build a Map<String, Integer>.
17. Convert a `List<Employee>` to `Map<Integer, Employee>` and detect if any id is duplicated (should throw).

Partition (Pattern 6)

18. Partition employees into those earning above vs below 60000.
19. Partition a list of integers into primes and non-primes.
20. Partition words into palindromes and non-palindromes.

Flatten (Pattern 7)

21. Given `List<List<Integer>>`, flatten into a single sorted `List<Integer>` with duplicates removed.
22. Given a `List<Employee>` where each has `List<String> skills`, get the distinct set of all skills across the company.
23. Given a `Map<String, List<Employee>>` (dept → employees), flatten back into a single sorted-by-salary `List<Employee>`.
24. Split a sentence into words, then flatten into all distinct characters used.

Top-N / Kth (Pattern 8)

25. Find the second highest salary (classic — you've done this).
26. Find the 3rd lowest score in a list of exam scores.
27. Find the top 3 most frequent words in a list of strings.
28. Find the youngest employee in each department (bottom-1 per group).

Reduction (Pattern 9)

29. Find the total number of characters across all words in a list.
30. Find the longest word in a list of strings.
31. Concatenate all employee names into one string using `reduce` (not `joining`) — then explain why `joining` is actually better here.
32. Find the product of all numbers in a list using `reduce`.
33. Find the employee with the maximum (salary - age) using a custom Comparator.

Sorting (Pattern 10)

34. Sort employees by department, then by salary descending within each department.
35. Sort a list of strings by length, then alphabetically for ties.
36. Sort a `Map<String, List<Employee>>` by the size of its value lists, descending.

Distinct-by-key (Pattern 11)

37. Given employees where `equals()` is NOT overridden, return distinct employees by id.

38. Given a list of orders with a `customerId`, get one order per customer (the first one seen).

Stateful-scan recognition (Pattern 12)

39. Find the first repeating character (again — but justify in one line why this is NOT a good pure-stream candidate).
40. Compute a running total (cumulative sum) of a list of numbers — explain why streams alone can't cleanly do this without external state.
41. Find the longest substring without repeating characters (classic — identify why this needs a loop/sliding window, not a stream one-liner).

String joining (Pattern 13)

42. Join employee names into `"Alice, Bob, Carol"`.
43. Join department names into `"[HR | Eng | Sales]"` using custom prefix/suffix/delimiter.

Match / Count (Pattern 14)

44. Check if any employee earns more than 100000.
45. Check if all employees are above 30000 salary.
46. Count how many words in a list start with a vowel.

Summary statistics (Pattern 15)

47. Get min, max, average, and count of salaries in a single pass.
48. Get summary statistics of word lengths in a list of strings.

Primitive streams (Pattern 16)

49. Given `int[] arr`, find the sum and average without boxing to `Integer`.

Collect into structure (Pattern 17)

50. Return department names as a `TreeSet` (sorted, no duplicates).

Answer Key — pattern + collector only

1–10: groupingBy (+ counting / averagingDouble / summingDouble / maxBy / minBy as needed)
11–13: groupingBy(Function.identity(), LinkedHashMap::new, counting()) 14–17: toMap (watch

for duplicate-key merge function) 18–20: partitioningBy 21–24: flatMap + distinct/sorted 25–28: distinct().sorted().skip/limit, or groupingBy + minBy/maxBy per group 29–33: reduce / mapToInt.sum / max(Comparator) 34–36: sorted(Comparator.comparing(...).thenComparing(...)) 37–38: toMap(keyFn, identity, (a,b) -> a, LinkedHashMap::new).values() 39–41: recognize — loop or sliding window, NOT a forced stream one-liner 42–43: Collectors.joining(delimiter, prefix, suffix) 44–46: anyMatch / allMatch / filter+count 47–48: summaryStatistics() 49: IntStream (mapToInt / Arrays.stream(int[])) 50: collect(Collectors.toCollection(TreeSet::new))

How to use this set effectively

- Don't run all 50 in one sitting — do 5/day, timed (5–8 min each), matching real interview pressure.
- After solving, say the pattern name and collector out loud before checking the answer key — that's the actual skill being drilled, not the code.
- Redo questions 1, 11, 25, 39 (the "anchor" questions) cold after a week — these are the ones interviewers riff on most.
- Once comfortable, retry #39–41 by explicitly forcing a stream solution, then argue against your own code — this is a strong differentiator in interviews.